Make Attention Sub-Quadratic Again

Learned ANN-Compatible Search Projections for Post-Hoc Sparse Attention

Marcel Butucea
Independent Researcher

May 2026

Abstract

Full self-attention gives decoder-only language models their long-range associative power, but its candidate-scoring cost grows quadratically with sequence length. Recent systems reduce this burden with optimized exact kernels, fixed sparse patterns, query-aware page selection, linear attention approximations, or retrieval-based attention over the native key/value cache. However, vanilla approximate-nearest-neighbor (ANN) search over a transformer's native query and key vectors is poorly matched to the geometry of attention: queries and keys are trained to interact through a dot product inside a layer, not necessarily to occupy a shared retrieval space.

This paper proposes a small trainable search layer for sub-quadratic attention candidate selection. For selected frozen transformer layers, we train per-layer query and key projections into a low-dimensional search space. These projections are distilled from the teacher model's own attention distributions using teacher-topK contrastive learning plus distribution-level KL distillation. At inference, dense candidate scoring over all prior keys is replaced by retrieving K candidate keys in the learned search space, followed by ordinary scaled-dot-product attention over only those candidates. The base model weights are never modified.

On a six-layer pilot using Qwen3-4B-Instruct-2507 with 4K context and WikiText-103, a clean block-causal $d_{\text{search}} = 128$ run preserves full-attention perplexity within +0.07% at $K = 128$ and +0.01% at $K = 256$. Learned search captures more teacher-attention mass than a Quest-style page baseline at equal token budget, although paired perplexity is tied with Quest at $K = 256$ and slightly worse at $K = 128$. A CPU FAISS/HNSW prototype tracks exact learned retrieval on the clean slice, demonstrating ANN compatibility but not production wall-clock speedup.

New broad-layer experiments show that substitution coverage is a quality/speed knob rather than a binary decision. Substituting all 36 layers is feasible but costs quality: the best all-36 checkpoint observed so far is step 500 with a +3.23% relative PPL gap, and per-layer retrieval diagnostics identify layers 0–2 as the weakest. Reserving layers 0, 1, 2, and 35 while substituting layers 3–34 improves the result substantially: the 32-layer run converges by step 500 and finishes at step 1000 with a +1.746% training-eval PPL gap, Recall@K = 0.825, and 20.97M trainable parameters. A post-hoc exact K-sweep on the final checkpoint gives a +0.590% PPL gap at $K = 128$ and −0.062% at $K = 256$ on a two-batch clean block-causal slice. The central claim is narrow and falsifiable: learned search projections can make attention-relevant key selection compatible with standard ANN retrieval, preserving model quality in a clean pilot and enabling near-parity broad-layer substitution while replacing dense candidate scoring with a sub-quadratic retrieval interface.

Keywords: efficient attention, sparse attention, approximate nearest neighbor search, long-context inference, distillation, transformer inference, HNSW, FAISS.

Main evidence at a glance

Six-layer clean pilot	Near full-attention parity on clean block-causal WikiText-103: +0.07% PPL gap at $K = 128$ and +0.01% at $K = 256$.
Quest comparison	Learned search captures more teacher-attention mass, but Quest is slightly better at $K = 128$ PPL and tied at $K = 256$.
ANN compatibility	Per-segment CPU FAISS/HNSW tracks exact learned retrieval closely; this validates the representation, not runtime speed.
Broad substitution	All36 is feasible but costs +3.23% PPL at the best checkpoint. Reserving layers $0, 1, 2, 35$ gives a stronger all32 result: +1.746% training-eval PPL gap, and +0.590% at $K = 128$ / −0.062% at $K = 256$ on the exact sweep.

Scope of the claim

This is a research-prototype result. The paper claims ANN-compatible retrieval fidelity and near-parity sparse substitution on clean slices. It does *not* claim measured wall-clock speedup, decode-mode KV-cache integration, or production readiness.

1 Introduction

Scaled dot-product attention computes, for a sequence of length N,

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right), \qquad O = AV, \tag{1}$$

where Q, K, and V are query, key, and value tensors, d_h is the head dimension, and M is a causal or block-causal mask. The matrix $QK^\top$ contains $O(N^2)$ pairwise scores. Optimized exact kernels such as FlashAttention [3] reduce memory traffic and improve practical throughput, but they do not change the logical candidate set: each query is still scored against every eligible key.

Long-context inference exposes this weakness. For each new token, attention over a long KV cache requires reading, scoring, and softmaxing over a growing number of prior keys. Empirically, attention distributions are often sparse: only a small fraction of the KV cache dominates the output for a given query. This suggests a different design: retrieve a small set of relevant keys, then compute exact attention only on that set. The hard part is not the sparse attention computation once candidates are known; it is finding the right candidates cheaply and reliably.

A tempting approach is to build an ANN index over native key vectors and retrieve nearest keys for each native query vector. RetrievalAttention [6] shows why this fails in practice. Native query and key vectors are not trained as symmetric nearest-neighbor embeddings. They are trained as two sides of a dot-product scoring system inside a transformer layer. The query/key distribution mismatch makes off-the-shelf ANN search over native Q/K unreliable unless the index itself becomes attention-aware.

This work takes the opposite route. Instead of modifying the index to compensate for hostile native vectors, it learns a small search space in which queries and keys are explicitly trained to be mutually retrievable. For each substituted layer i, two lightweight projections map hidden states to low-dimensional search vectors:

$$q_i^s = W_{Qs}^i h_i, \qquad k_i^s = W_{Ks}^i h_i, \tag{2}$$

with $q_i^s, k_i^s \in \mathbb{R}^{d_{\text{search}}}$. Retrieval is performed in this learned space, while the final attention output still uses the frozen model's native Q, K, and V restricted to the retrieved key set.

Contributions. The contribution is deliberately modest:

1. A distillation objective that trains low-dimensional per-layer search projections to recover attention-relevant keys from a frozen teacher model.
2. A clean block-causal evaluation showing near-parity perplexity when substituting learned sparse attention into six layers of Qwen3-4B-Instruct-2507.
3. A comparison against a Quest-style page selector showing higher teacher-attention mass for learned retrieval, but no clean perplexity advantage on the current slice.
4. A FAISS/HNSW correctness prototype showing that the learned vectors are compatible with an off-the-shelf ANN index.
5. Broad-layer evidence: all-36 substitution is feasible but not parity, while reserving the weak edge layers $\{0, 1, 2, 35\}$ gives a much smaller quality gap with 32/36 layers substituted.
6. A clear separation between the current evidence and the missing engineering: no measured wall-clock speedup, no decode-mode KV-cache integration, no long-context task validation, and no multi-seed confidence intervals yet.

2 Background and Design Target

2.1 Full attention and candidate scoring

For a single head and query position t, full causal attention computes scores against every valid key position $j \le t$:

$$s_{tj} = \frac{q_t^\top k_j}{\sqrt{d_h}}, \qquad \alpha_{tj} = \frac{\exp(s_{tj})}{\sum_{\ell \le t} \exp(s_{t\ell})}, \qquad o_t = \sum_{j \le t} \alpha_{tj} v_j. \tag{3}$$

The per-query scoring cost is $O(Nd_h)$, and the full prefill matrix costs $O(N^2 d_h)$. Reducing attention cost requires either reducing the number of scored keys, making scoring more IO-efficient, or both.

2.2 Sparse and retrieval-based attention

Sparse attention replaces the full candidate set with a smaller set S_t:

$$o_t = \sum_{j \in S_t} \mathrm{softmax}_{j \in S_t}\left(\frac{q_t^\top k_j}{\sqrt{d_h}}\right) v_j. \tag{4}$$

The quality of the method depends on whether S_t contains the keys with high teacher-attention mass. This paper uses two retrieval-quality metrics:

$$\mathrm{Recall@K}(t) = \frac{|T_K(t) \cap S_K(t)|}{\min(K, |C_t|)}, \tag{5}$$

where $T_K(t)$ is the teacher top-K set and C_t is the valid causal key set, and

$$\mathrm{Mass@K}(t) = \sum_{j \in S_K(t)} \alpha_{tj}^{\mathrm{teacher}}. \tag{6}$$

Mass@K is often more informative than Recall@K when teacher attention is sharp, because missing a low-probability topK key is less damaging than missing a high-probability key.

2.3 Quest and RetrievalAttention

Quest [11] uses query-aware page selection over the KV cache. Each page stores min/max key metadata; a query estimates page importance and loads only the top pages for attention. Quest is attractive because it is training-free, hardware-aware, and directly targets KV-cache memory movement. In this work, Quest is treated as the strongest practical heuristic baseline currently implemented in the repository. The comparison here is not against the original optimized Quest runtime; it is against a correctness-oriented Quest-style page selector using the same block-causal mask and sparse gather path as learned retrieval.

RetrievalAttention [6] directly addresses long-context attention with vector retrieval over the KV cache. Its key observation is central to this work: off-the-shelf ANN indexes over native Q/K vectors are ineffective because query and key distributions are out of distribution with respect to each other. RetrievalAttention handles this by making the search algorithm attention-aware. This paper instead changes the vectors. The learned projections produce q^s and k^s in a shared retrieval space, so standard ANN machinery can be used without making the index attention-aware. The trade-off is training: RetrievalAttention is training-free; this method adds a small number of trainable projection parameters.

2.4 The intended deployment regime

The method is not designed to beat Quest at every context length. At 4K or 32K, a page heuristic can be cheaper than HNSW-style retrieval. The target regime is very long context, where linearly scanning keys or pages becomes unattractive, and where a dynamic ANN index can avoid dense candidate scoring. The current paper validates the representation and substitution side of this idea. It does not yet validate a production runtime.

3 Method

3.1 Frozen teacher and trainable search projections

Let f be a frozen decoder-only transformer. For a selected layer i, let $h_i \in \mathbb{R}^{B\times N\times d_{model}}$ be the hidden state entering the layer's self-attention module after input layer normalization. The method attaches a search projection module to that layer:

$$Q_i^s = h_i W_{Qs}^i, \qquad K_i^s = h_i W_{Ks}^i, \tag{7}$$

where

$$W_{Qs}^i, W_{Ks}^i \in \mathbb{R}^{d_{model}\times d_{\text{search}}}. \tag{8}$$

The pilot uses linear projections. For Qwen3-4B-Instruct-2507, $d_{model} = 2560$. With $d_{\text{search}} = 128$ and six trained layers, the search module contains approximately 3.93M trainable parameters. With 32 substituted layers, it contains 20.97M parameters. The base model remains frozen.

3.2 Teacher attention reconstruction

Training requires the teacher's attention distribution. A naive implementation would request attention outputs directly, but that can force attention layers onto slower eager paths. The implementation instead captures native post-RoPE Q and K tensors for selected layers and reconstructs the teacher distribution outside the model forward:

$$A_i^T = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_h}} + M\right). \tag{9}$$

For grouped-query attention, KV heads are repeated to match query heads before reconstruction. The teacher attention is then aggregated across heads, using max aggregation in the current configuration:

$$\bar{A}_i^T[t,j] = \max_h A_i^T[h,t,j]. \tag{10}$$

The implementation includes a QK reconstruction verification test that checks agreement between reconstructed and eager attention rankings. If the reconstructed teacher rankings are wrong, the search projections are trained against the wrong supervision signal.

3.3 Contrastive teacher-topK objective

For each query token t, the teacher distribution selects a positive set P_t consisting of the top K_{pos} teacher keys. Search vectors are L2-normalized for the contrastive loss:

$$\tilde{q}_t^s = \frac{q_t^s}{\|q_t^s\|_2}, \qquad \tilde{k}_j^s = \frac{k_j^s}{\|k_j^s\|_2}. \tag{11}$$

The search similarity is

$$z_{tj} = \frac{(\tilde{q}_t^s)^\top \tilde{k}_j^s}{\tau}. \tag{12}$$

The multi-positive InfoNCE loss is

$$\mathcal{L}_{\text{NCE}}(t) = -\log \frac{\sum_{j \in P_t} \exp(z_{tj})}{\sum_{j \in C_t} \exp(z_{tj})}, \tag{13}$$

where C_t is the valid causal or block-causal key set. Queries with too few valid keys and padded query positions are excluded.

3.4 Distribution-level distillation

The second loss aligns the full search distribution with the teacher distribution. Search logits are computed without L2 normalization:

$$r_{tj} = \frac{(q_t^s)^\top k_j^s}{\sqrt{d_{\text{search}}}}. \tag{14}$$

The student distribution is

$$A_i^S[t, j] = \text{softmax}_{j \in C_t}(r_{tj}/\tau_d). \tag{15}$$

The distillation loss is

$$\mathcal{L}_{\text{KL}}(t) = D_{KL}\left(A_i^T[t, \cdot] \parallel A_i^S[t, \cdot]\right) = \sum_{j \in C_t} A_i^T[t, j] \log \frac{A_i^T[t, j]}{A_i^S[t, j]}. \tag{16}$$

The total layer-averaged objective is

$$\mathcal{L} = \alpha \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \mathcal{L}_{\text{NCE}}^i + \beta \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \mathcal{L}_{\text{KL}}^i, \tag{17}$$

with $\alpha = \beta = 1$ in the pilot.

3.5 Inference-time substitution

At inference, selected attention layers are monkey-patched. For a substituted layer:

1. Compute native Q, K, and V exactly as the base model would, including q-norm/k-norm and RoPE.
2. Compute Q^s and K^s from the same hidden states.
3. Retrieve K_{retrieve} candidate keys per query in the learned search space.

4. Gather native K and V at those retrieved positions.
5. Compute normal scaled-dot-product attention over the gathered candidate set only.
6. Apply the original output projection.

The final sparse attention is not an approximation in value space. The approximation is candidate selection only. For query t and retrieved set S_t, the substituted attention output is

$$\hat{o}_t = \sum_{j \in S_t} \frac{\exp(q_t^\top k_j / \sqrt{d_h})}{\sum_{\ell \in S_t} \exp(q_t^\top k_\ell / \sqrt{d_h})} v_j. \tag{18}$$

3.6 Block-causal packing

The initial packed experiments were leakage-confounded: tokens from different packed documents could attend across document boundaries. The clean evaluation uses segment-level block-causal masking. Each packed document receives a segment id, position ids are reset at segment boundaries, and the attention mask allows a query token to attend only to earlier tokens in the same segment:

$$M_{tj} = 0 \quad \text{iff} \quad \text{segment}(t) = \text{segment}(j) \text{ and } j \le t, \tag{19}$$

and $M_{tj} = -\infty$ otherwise. Retrieval, loss masking, Recall@K, Mass@K, and sparse attention all use this same eligibility relation.

4 Complexity and Deployment Model

This section analyzes candidate scoring, not the full attention layer. Once K candidates are selected, sparse attention still costs $O(Kd_h)$ per query for native scoring and value aggregation.

For full attention, the per-query candidate-scoring proxy is

$$C_{full}(N) = Nd_h. \tag{20}$$

With $d_h = 128$, $C_{full}(N) = 128N$.

For Quest-style page selection with page size $P = 16$, using min/max metadata over native keys,

$$C_{Quest}(N) = \frac{N}{P} \cdot 2d_h = 16N. \tag{21}$$

For learned HNSW retrieval, using $M = 32$, $ef_{search} = 64$, and $d_{\text{search}} = 128$, the deterministic operation-count proxy used in the repository is

$$C_{HNSW}(N) = M \cdot ef_{search} \cdot \log_2(N) \cdot d_{\text{search}}. \tag{22}$$

This gives

$$C_{HNSW}(N) = 32 \cdot 64 \cdot 128 \cdot \log_2(N). \tag{23}$$

This is not a measured GPU runtime model and should not be read as a guarantee for HNSW. It is a candidate-scoring proxy. Under these constants, Quest is cheaper below the few-hundred-thousand-token regime, while learned HNSW becomes cheaper at very long contexts. The repository's deterministic scaling table places the rough crossover against Quest around the 300K-token range and gives an approximately 3× scoring-proxy advantage for learned HNSW at 1M context.

The practical implication is sharp: this method is not automatically a faster replacement for Quest at 4K or 32K context. The argument becomes interesting when context length is very large and candidate selection must avoid linear scans over pages or tokens.

5 Experimental Setup

5.1 Model

The experiments use Qwen/Qwen3-4B-Instruct-2507, a 36-layer causal language model with 4.0B parameters, 32 query heads, 8 KV heads, grouped-query attention, and a native 262,144-token context length [9]. The six-layer pilot substitutes

$$\mathcal{I} = \{4, 8, 12, 16, 20, 24\}. \tag{24}$$

The broad runs substitute either all 36 layers or layers 3–34 while reserving layers 0, 1, 2, and 35 as full attention.

5.2 Data and training

The training and evaluation data for the reported prototype runs is WikiText-103 raw text [8]. The sequence length is 4096. The six-layer runs use batch size 8 and train for 1000 steps. The all-36 and all32 broad runs use a smaller batch with gradient accumulation because QK reconstruction and teacher attention capture are more expensive when many layers are trained. Search projections are trained while all base-model weights remain frozen.

5.3 Metrics

The paper reports:

- Perplexity under full attention and substituted sparse attention.
- Relative PPL gap: $(\mathrm{PPL}_{sparse} - \mathrm{PPL}_{full})/\mathrm{PPL}_{full}$.
- Recall@K against teacher topK.
- Mass@K, the teacher probability mass captured by the retrieved set.
- FAISS filler rate for the prototype ANN path.
- Paired NLL deltas for learned-vs-Quest comparison where available.

PPL is the primary model-quality metric. Mass@K is the primary retrieval-fidelity metric. Recall@K is reported, but it is less reliable when teacher attention has a long low-probability tail.

6 Results

6.1 Capacity ablation on packed WikiText-103

The packed d_{search} ablation should be interpreted as a capacity comparison, not as the clean headline result, because the original packed path had cross-document leakage. At $K = 128$, capacity scaling is monotonic but saturating (Table 1). $d_{\text{search}} = 128$ captures almost all of the $d_{\text{search}} = 256$ retrieval quality at half the trainable parameter count, so $d_{\text{search}} = 128$ is the recommended pilot capacity.

Table 1. Packed WikiText-103 capacity ablation at $K = 128$. This is a capacity comparison; the clean quality claim uses block-causal masking.

d_{search}	Params	Learned Mass@K	Raw-QK mass	Learned/raw	Final PPL gap
64	1.97M	0.492	0.488	1.01×	+2.39%
128	3.93M	0.503	0.488	1.03×	−1.81%
256	7.86M	0.509	0.488	1.04×	−1.85%

6.2 Clean six-layer block-causal run

The clean block-causal run removes cross-document leakage. On the 16-batch clean evaluation slice, full-attention PPL is 30.44. The substituted path is effectively at full-attention parity (Table 2). K=512 has no meaningful average recall/mass on this WikiText slice because almost no same-segment queries have 512 valid causal keys, but filler slots are masked out of the sparse-attention softmax and the PPL line remains useful.

Table 2. Clean six-layer block-causal $d_{\text{search}} = 128$ exact retrieval sweep.

K	Recall@K	Mass@K	Sparse PPL	Relative PPL gap
128	0.744	0.787	30.47	+0.07%
256	0.879	0.953	30.45	+0.01%
512	n/a	n/a	30.45	+0.01%

Clean block-causal per-layer Mass@K at $K = 128$ is shown in Table 3. With segment isolation, early trained layers are not uniquely hard; all six tested layers have high oracle mass, and the learned projection matches or slightly exceeds raw-QK retrieval across the tested set.

Table 3. Clean block-causal per-layer Mass@K at $K = 128$.

Layer	Raw-QK oracle mass	Learned d128 mass
4	0.956	0.950
8	0.977	0.976
12	0.970	0.977
16	0.964	0.970
20	0.970	0.983
24	0.978	0.984
Average	0.969	0.973

6.3 Quest-style page baseline

A Quest-style min/max page baseline was evaluated with page size 16, native post-RoPE Q/K, the same block-causal token eligibility mask, and the same sparse-attention gather path. Learned search recovers more teacher-attention mass at equal K, especially at $K = 128$ (Table 4). However, perplexity does not currently show a clean advantage over Quest.

Table 4. Learned search vs. Quest-style page baseline on the same clean block-causal eval slice.

Method	K	Recall@K	Mass@K	PPL gap
Learned search exact	128	0.744	0.787	+0.07%
Quest-style page	128	0.669	0.727	−0.11%
Learned search exact	256	0.879	0.953	+0.01%
Quest-style page	256	0.838	0.909	+0.03%

A paired 32-batch NLL evaluation gives the sharper reading (Table 5). The honest claim is not that learned search beats Quest in PPL. The current claim is that learned search provides higher retrieval fidelity and ANN compatibility; PPL is tied with Quest at $K = 256$ and slightly worse at $K = 128$ on the paired WikiText slice.

Table 5. Paired learned-vs-Quest NLL comparison. Positive learned-minus-Quest means learned search is worse.

K	Full PPL	Learned PPL	Quest PPL	Learned - Quest NLL delta, 95% bootstrap CI
128	28.03	28.07	28.01	+0.00205 [+0.00160, +0.00251]; Quest slightly better
256	28.03	28.04	28.04	−0.00005 [−0.00029, +0.00018]; statistical tie

6.4 FAISS/HNSW compatibility

The first block-causal FAISS prototype used a global index followed by segment filtering, which caused pathological filler rates. The corrected clean path builds per-segment indexes when a block-causal mask is present. With that fix, CPU FAISS/HNSW tracks exact learned search on the same clean 16-batch evaluation slice (Table 6). The filler rate is expected for short same-segment prefixes where fewer than K valid causal keys exist; filler slots are masked out of the sparse-attention softmax. These results demonstrate compatibility with off-the-shelf ANN retrieval, not production efficiency.

Table 6. Clean exact-vs-FAISS check.

Method	K	PPL gap	FAISS filler rate
Learned exact	128	+0.07%	n/a
Learned FAISS/HNSW	128	+0.09%	0.447
Learned exact	256	+0.01%	n/a
Learned FAISS/HNSW	256	+0.04%	0.683

6.5 Dynamic-index proxy

The current ANN wrapper is prefill-only with `use_cache=False`. A true generation-time dynamic-index benchmark requires KV-cache integration. As a first capability proxy, the dynamic-index experiment splits clean block-causal sequences into a prefill prefix and a decode-like suffix. It compares dynamic retrieval, where suffix queries can retrieve from all same-segment prior keys, against a static index where suffix queries can retrieve from prefill keys plus a 256-token recent local suffix window but not older suffix keys. At $K = 128$, prefill length 1024, local window 256, and 8 eval batches, the dynamic proxy captures more teacher mass (Table 7). This does not establish task accuracy or decode latency. It only shows that a frozen prefill-plus-local index loses measurable teacher-attention mass on decode-like suffix queries.

Table 7. Dynamic-index proxy at $K = 128$.

Setting	Teacher mass captured
Dynamic proxy	0.972
Static proxy	0.928
Static teacher mass available	0.954
Dynamic - static	+0.044

6.6 All-36 and all32 reserved-layer substitution

After the six-layer clean pilot, a stronger full-substitution experiment trained $d_{\text{search}} = 128$ projections for all 36 attention layers under the same packed block-causal methodology. The all-36 result is feasible but not a parity result. Broad substitution preserves substantial retrieval fidelity, but per-layer approximation errors compound into a +3–4% PPL gap. Step 500 is the best checkpoint observed by PPL (Table 8). Steps 500, 750, and 800 have essentially identical Mass@K, so the non-monotonic PPL trajectory is not explained by a large retrieval-mass change.

Table 8. All-36 $d_{\text{search}} = 128$ training trajectory.

Step	Recall@K eval	Relative PPL gap	Read
250	0.805	+6.271%	ranking still poor
500	0.816	+3.227%	best observed checkpoint
750	0.817	+3.964%	PPL regressed despite stable recall

The diagnostic is clear: the weak layers are concentrated at the beginning of the network. Layer 0 is the main outlier, layer 1 is weak, and layer 2 is borderline. Layer 35 is mildly below raw-QK but not the primary failure mode. This motivated the reserved-edge follow-up: reserve layers 0, 1, 2, and 35 as full attention and train layers 3–34. That run substitutes 32 of 36 layers, trains 20.97M search-projection parameters, and keeps only four layers as full attention.

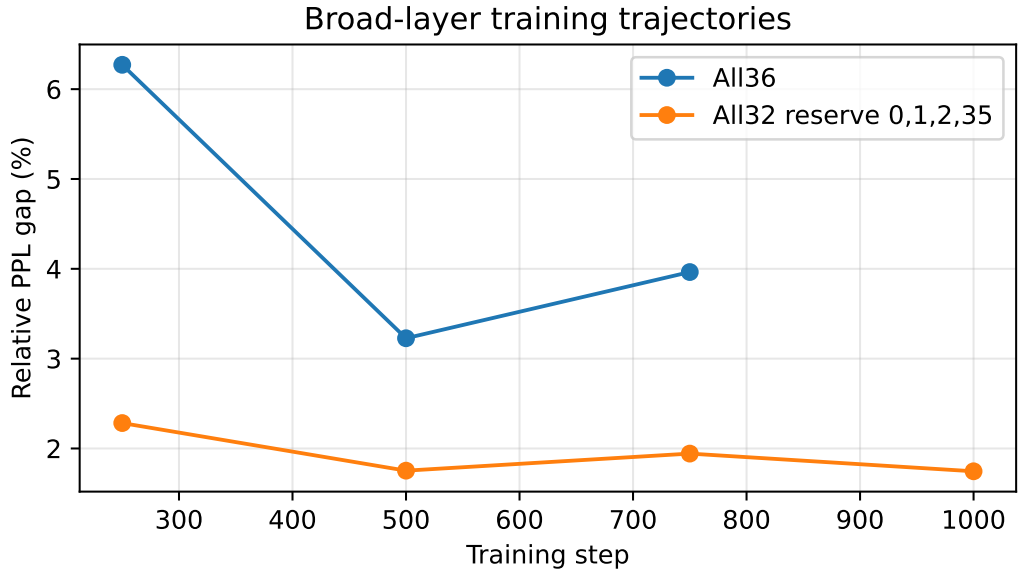

Figure 1. Broad-layer training trajectories. The all32 reserved-edge configuration cuts the all-36 quality cost roughly in half while preserving 89% layer coverage.

The all32 result is stable rather than peak-then-regress: step 1000 is essentially identical to step 500 (Table 9). This is the current broad-substitution headline. It is not full parity at $K = 128$ in the training eval, but it is a large improvement over the all-36 run while retaining 89% layer coverage.

Table 9. All32 reserved-edge training trajectory. Layers 0, 1, 2, and 35 remain full attention; layers 3–34 are substituted.

Step	Recall@K eval	Relative PPL gap	Read
250	0.812	+2.283%	already better than all36 best training eval
500	0.823	+1.753%	converged to final quality band
750	0.825	+1.943%	small eval fluctuation
1000	0.825	+1.746%	final checkpoint; essentially tied with step 500

Post-hoc compare-retrieval on step 1000 shows that, on the substituted layers, learned retrieval matches raw-QK mass: learned Mass@K=128 is 0.971 versus 0.969 for raw-QK, and learned Mass@K=256 is 0.993 versus 0.994 for raw-QK. The edge-layer reservation therefore removes the main per-layer retrieval outliers; the remaining quality cost is more plausibly a compound approximation effect.

The exact K-sweep clarifies the deployment knob (Table 10). At $K = 128$, 32-layer substitution is near parity on the exact sweep; at $K = 256$, it is statistically indistinguishable from full attention on this small slice. Lower K values expose the expected quality cost. K=512 is intentionally omitted: the script produced a valid sparse-attention PPL line but zero Mass@K/Recall@K due to an edge-case bug in metric handling when K exceeds the number of valid causal keys for most same-segment queries.

Table 10. All32 step-1000 exact K-sweep on a two-batch clean block-causal slice, with $\mathrm{PPL}_{full} = 20.5349$.

K	Mass@K	Recall@K	Sparse PPL	Relative PPL gap
16	0.546	0.518	24.86	+21.064%
32	0.627	0.572	21.85	+6.422%
64	0.722	0.652	20.94	+1.974%
128	0.807	0.746	20.66	+0.590%
256	0.902	0.876	20.52	−0.062%

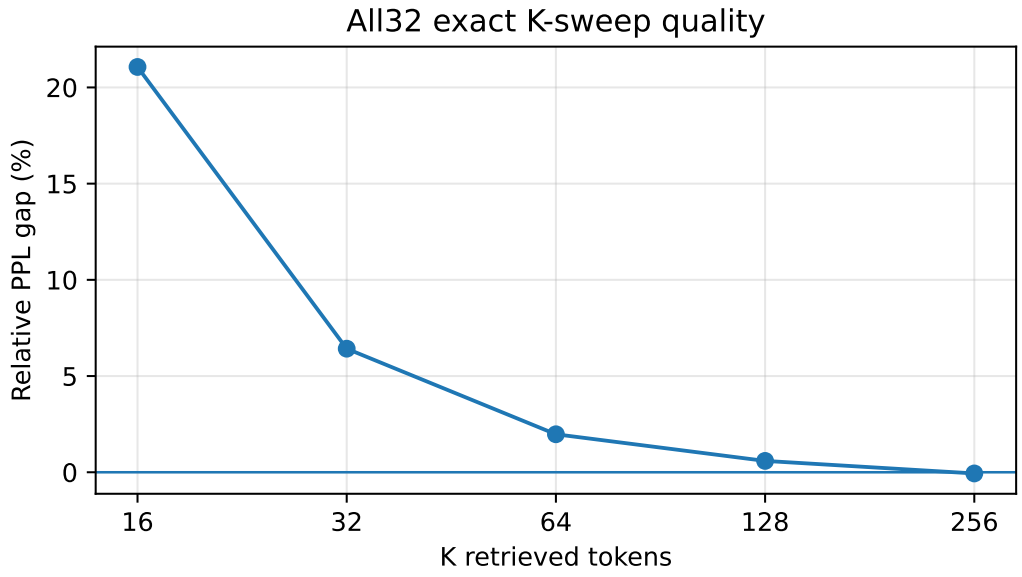

Figure 2. All32 exact K-sweep PPL gap. The quality cliff is sharp at low K, but $K = 128$ is near parity and $K = 256$ is effectively tied with full attention on this small slice.

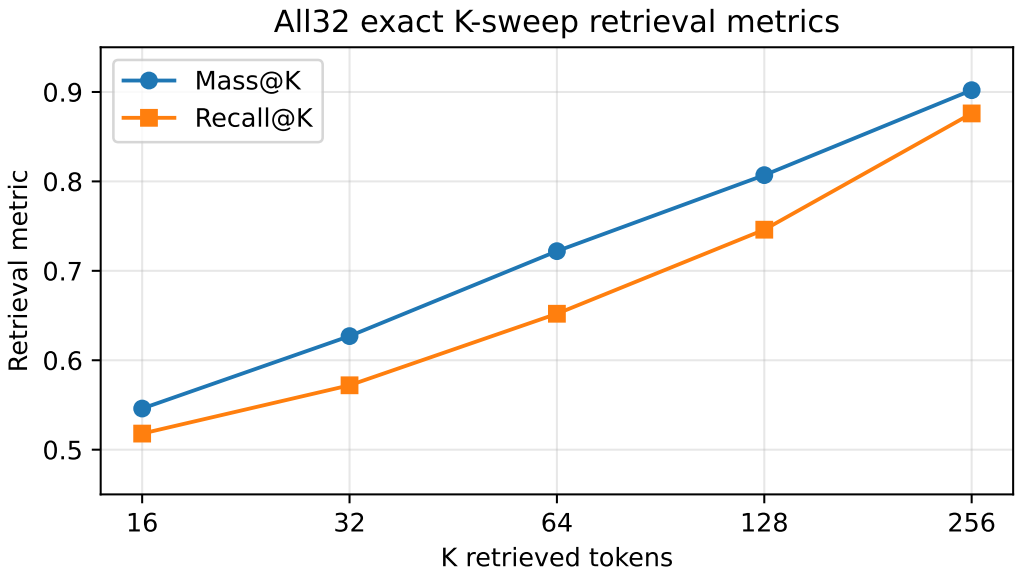

Figure 3. All32 exact K-sweep retrieval metrics. Mass@K and Recall@K both rise smoothly with K, consistent with the PPL trend.

6.7 Coverage as a Pareto knob

The broad-layer results suggest that substitution coverage is itself an important hyperparameter. The current coverage points are shown in Table 11. They come from different eval slices and should be read as a qualitative Pareto curve, not a single controlled sweep.

Table 11. Layer coverage vs. quality cost. Values come from different eval slices and are a qualitative Pareto sketch, not a controlled sweep.

Configuration	Layers substituted	Coverage	PPL gap	Interpretation
6-layer clean pilot	6/36	17%	+0.07% at $K = 128$	Quality-preserving pilot
All32 reserved-edge	32/36	89%	+1.746% train; +0.590% exact	Near-parity broad substitution
All36	36/36	100%	+3.227% best	Full substitution costs quality

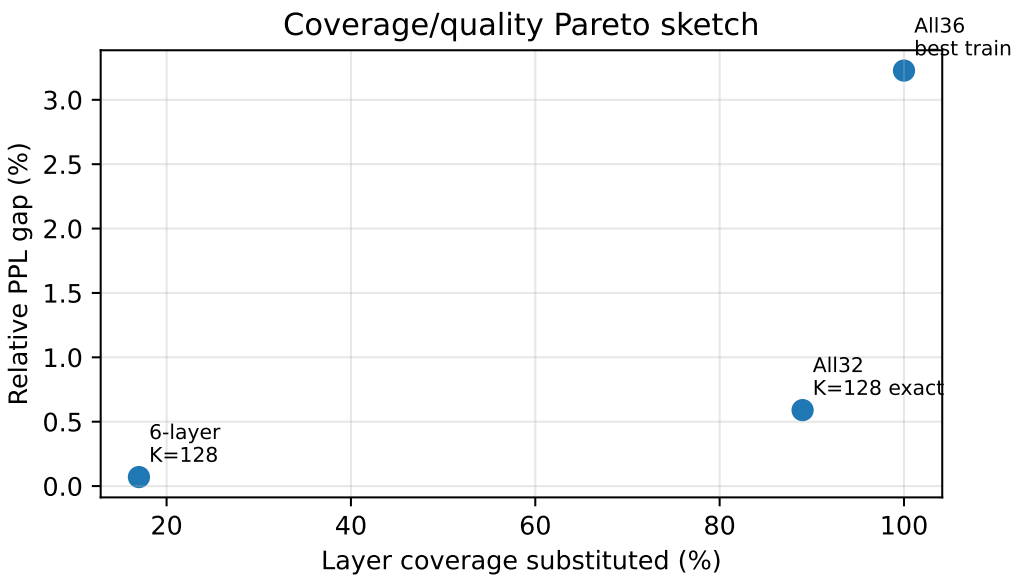

Figure 4. Coverage/quality Pareto sketch. The likely best deployment point is not simply "sparsify everything"; edge layers matter, and approximation errors compound.

This is not enough to claim an optimal coverage ratio, but it strongly suggests that the best deployment point is intermediate rather than full substitution. The next clean experiment is a controlled coverage sweep, for example 12-, 18-, and 20-layer configurations, to test whether the zero-gap point lies around the 50–60% coverage range implied by simple interpolation between the current points.

7 Discussion

7.1 What the result proves

The clean result proves that learned low-dimensional search projections can preserve the quality of selected transformer layers when sparse attention candidates are selected in the learned space. It also shows that standard HNSW retrieval can approximate exact learned retrieval closely enough on the clean slice. This separates two problems:

1. Can the model's attention-relevant keys be represented in a small shared retrieval space?
2. Can a production kernel retrieve from that space faster than dense attention or page selection?

The pilot gives evidence for the first question. It does not yet solve the second.

7.2 Decoupling training-time and inference-time architecture

Sliding-window attention, Longformer-style sparse attention, and state-space or hybrid models are usually deployed as native architectures: they are trained from scratch with the sparse or hybrid mechanism already in place. These approaches can achieve strong inference efficiency, but they require designing and training the model around the chosen attention pattern.

This work targets a different deployment scenario: enabling sparse attention on already-trained full-attention models without retraining the base. For pretrained models such as Qwen, Llama, and GPT-family models, where full retraining is expensive, post-hoc retrieval-based sparse attention provides a complementary path. Whether to design new models with native sparse attention or apply post-hoc retrieval to existing models is an architectural decision that depends on training budget and deployment constraints.

The broader motivation is to decouple training expressivity from inference efficiency. Full attention remains the safest training-time architecture for learning complex dependencies and reasoning patterns at frontier scale, while its efficiency disadvantages matter most during inference, where KV-cache traffic and candidate scoring constrain deployment economics. This method exploits that asymmetry: train with full attention for maximum expressivity, then add lightweight retrieval projections to enable sub-quadratic inference without modifying the base model's weights.

7.3 What the result does not prove

The current repository does not prove measured wall-clock speedup. The FAISS path builds CPU indexes per forward pass, performs GPU-to-CPU transfers, and uses dense-style tensor expansion internally for gather. It is a correctness prototype.

The current repository also does not prove full-model parity. The clean six-layer result is near parity, and the all32 reserved-edge run is near parity at $K = 128$ and parity at $K = 256$ on a small exact K-sweep, but the all-36 run observed so far has a +3–4% PPL gap. Full substitution remains a harder setting.

The current repository does not prove long-context task quality. WikiText perplexity is a useful first sanity check but is not a substitute for LongBench, RULER, passkey retrieval, needle-in-haystack, or downstream generation benchmarks.

Finally, the clean comparison does not beat Quest in perplexity. Learned search has higher Mass@K, but Quest is slightly better at $K = 128$ and tied at $K = 256$ on the paired NLL evaluation.

7.4 Why learned retrieval may still matter

Quest is a strong short- and medium-context baseline because page metadata gives a cheap query-aware score. Under the repository's scoring proxy, Quest is cheaper than learned HNSW below a few hundred thousand tokens. Learned ANN retrieval becomes interesting when:

- context length reaches hundreds of thousands to millions of tokens;
- the retrieval index can live on GPU or in a low-latency paged memory system;
- the index can be incrementally updated during decode;
- higher Mass@K translates into task quality at lower K or under harder long-context distributions.

The next experiments should therefore not merely repeat WikiText. They should test whether learned retrieval allows lower K than Quest on tasks with sparse, long-range dependencies.

7.5 Failure analysis from the all-36 run

The all-36 run is useful precisely because it failed to reach parity. It shows that high average Mass@K is not sufficient when every layer is substituted. Early layers appear more sensitive because they sit closest to lexical and positional representation formation. Approximation noise there can propagate through the entire stack. Reserving layers 0–2 removes the largest per-layer retrieval deficits. Reserving layer 35 is conservative: the last layer is close to logits, where errors are less recoverable. The all32 run indicates that edge-layer reservation is a strong first-order fix, but the remaining +1–2% training-eval gap shows that compound approximation still matters.

8 Related Work

8.1 Property-level comparison

The useful comparison is not only to Quest. Quest is the closest practical page-selection baseline, but Reformer is closer in architectural shape: candidate selection is performed by an ANN-like mechanism and attention is then computed over a restricted set [5]. Performer is also genuinely sub-quadratic, but it changes the attention computation itself by approximating the softmax kernel with random features [2]. Longformer and BigBird use sparse patterns that are cheap but mostly fixed rather than fully query-adaptive [1, 14]. NSA-style methods are natively trainable sparse-attention mechanisms with hardware-aligned block structure [13]. These distinctions matter because a long-context method can fail in different ways: it may be sub-quadratic but untrained, trained but still asymptotically dense, query-aware but linear in the number of pages, or fast but unable to represent sharp retrieval.

Table 12. Qualitative comparison of sparse and efficient attention mechanisms.

Method	Selection mechanism	Query-aware	Trained	Asymptotic	Exact softmax
Full attention	all keys	n/a	n/a	$O(N^2)$	yes
Reformer	LSH hashing	yes	no	$O(N \log N)$	over bucket
Performer	random features	n/a	no	$O(N)$	no
BigBird	window + random + global	mostly no	no	$O(N)$	over pattern
Longformer	sliding window + global	mostly no	no	$O(N)$	over pattern
NSA-style methods	block compression/selection	partial	partial	block-dependent	yes
Quest	min/max page heuristic	yes	no	$O(N)$ over pages	over selected pages
This work	trained low-dim retrieval	yes	yes	ANN target	over retrieved keys

8.2 Reformer as the closest asymptotic ancestor

Reformer's LSH attention can be viewed as untrained ANN-style candidate selection: hash the sequence, attend within nearby buckets, and avoid scoring every key. The appealing property is genuine sub-quadratic scaling. The weakness is that random hash functions do not know which keys the model's attention actually needs. Native Q/K vectors are also not guaranteed to be good nearest-neighbor embeddings. This work keeps Reformer's retrieve-then-attend shape but replaces untrained hashing with projections distilled from the teacher model's attention distribution.

8.3 Linear and smooth attention approximations

Performer approximates softmax attention with positive orthogonal random features, giving linear-time attention in principle. This is a different failure mode from sparse retrieval. Linear kernel approximations replace the softmax with a smoother low-rank computation, so they can lose the sharp selectivity needed for precise key-value retrieval. This method does not approximate the softmax inside the selected set; it preserves native scaled dot-product attention over retrieved keys. The approximation is only in candidate selection.

8.4 Fixed sparse patterns

Longformer and BigBird reduce cost with local windows, global tokens, random connections, or structured sparse patterns. These patterns are efficient and useful, but they are not fully query-aware in the sense used here: the selected remote keys are not chosen because a specific query token currently needs them. Query-aware methods such as Quest and learned retrieval are better matched to long-context retrieval tasks where the relevant token may be far away and content-dependent.

8.5 RetrievalAttention

RetrievalAttention is closest in motivation. It observes that native Q/K vectors are poorly suited for vanilla ANN due to query/key distribution mismatch, and it fixes this with attention-aware search. This paper fixes the mismatch by training a small shared search space. In one sentence: RetrievalAttention adapts the index to native Q/K; this work adapts Q/K-like search vectors to the index.

8.6 ANN indexing

HNSW and FAISS provide practical approximate nearest-neighbor search machinery [7, 4]. The current implementation uses FAISS HNSW as a correctness check. A deployable system would need a GPU-resident or cache-integrated index, not per-forward CPU index construction.

9 Limitations and Next Experiments

The following items are required before this work should be presented as more than a promising research prototype:

1. Multi-seed confidence intervals for the clean block-causal result.
2. Larger eval slices and stricter paired testing against Quest.
3. A four-way decomposition: full attention vs. exact native-QK topK vs. exact learned-search topK vs. FAISS learned-search topK.
4. Long-context task evaluation on LongBench, RULER, passkey retrieval, and needle-in-haystack.
5. A controlled coverage Pareto sweep, including intermediate 12-, 18-, and 20-layer configurations, to locate the maximum coverage point that preserves full-attention quality.
6. Decode-mode KV-cache integration with incremental index updates.
7. GPU-resident retrieval and fused sparse gather-attention kernels.
8. Measured wall-clock latency and memory footprint against FlashAttention/SDPA and optimized Quest.
9. Ablation over d_{search}, K_{pos}, KL-vs-contrastive weighting, head aggregation, and per-head vs. shared-head retrieval.
10. Cross-model validation beyond Qwen3-4B-Instruct-2507.

10 Conclusion

This paper proposes learning a small per-layer search space for attention candidate retrieval. The method keeps the base language model frozen, trains lightweight query/key search projections from teacher attention, retrieves candidate keys in the learned space, and performs normal attention over the retrieved native KV vectors.

The clean pilot result is encouraging but narrow: six substituted layers of Qwen3-4B-Instruct-2507 preserve WikiText perplexity under block-causal masking, and FAISS/HNSW closely tracks exact learned retrieval.

Compared with Quest, learned retrieval captures more teacher-attention mass but does not yet deliver a clean perplexity win. The first all-36 result shows that full substitution is feasible but still costs +3–4% relative PPL under the current hyperparameters. The all32 reserved-edge result is stronger: substituting layers 3–34 reaches a +1.746% PPL gap in training eval and +0.590% at $K = 128$ in a small exact K-sweep, with $K = 256$ effectively tied with full attention.

The result should therefore be framed as an ANN-compatibility, retrieval-fidelity, and coverage-Pareto contribution, not as a completed fast-attention system. The path to a stronger claim is clear: fill in the intermediate coverage curve, run long-context tasks, integrate decode-mode dynamic indexing, and demonstrate measured latency gains with a GPU-resident retrieval kernel. Until then, the title is aspirational but defensible: Make Attention Sub-Quadratic Again.

A All-36 Per-Layer Retrieval Diagnostics

Table 13 gives the all-36 step-500 per-layer Mass@K at $K = 128$. Delta is learned minus raw-QK. Layers 0, 1, and 2 are the largest negative outliers.

Table 13. All-36 step-500 per-layer Mass@K at $K = 128$.

Layer	Raw	Learned	Delta	Layer	Raw	Learned	Delta
0	0.922	0.780	−0.142	18	0.965	0.972	+0.007
1	0.918	0.851	−0.067	19	0.961	0.968	+0.007
2	0.939	0.899	−0.040	20	0.959	0.975	+0.016
3	0.939	0.924	−0.015	21	0.966	0.979	+0.014
4	0.944	0.933	−0.011	22	0.963	0.970	+0.007
5	0.964	0.947	−0.017	23	0.979	0.984	+0.005
6	0.956	0.936	−0.020	24	0.971	0.978	+0.007
7	0.982	0.982	+0.000	25	0.986	0.988	+0.002
8	0.971	0.970	−0.001	26	0.978	0.985	+0.008
9	0.959	0.976	+0.017	27	0.978	0.983	+0.005
10	0.974	0.970	−0.004	28	0.979	0.985	+0.005
11	0.976	0.975	−0.001	29	0.982	0.987	+0.005
12	0.961	0.969	+0.008	30	0.988	0.986	−0.002
13	0.971	0.971	+0.000	31	0.984	0.984	−0.001
14	0.973	0.973	−0.000	32	0.979	0.979	+0.000
15	0.968	0.972	+0.004	33	0.977	0.970	−0.007
16	0.956	0.962	+0.006	34	0.976	0.960	−0.016
17	0.959	0.966	+0.007	35	0.980	0.967	−0.013
			Average		0.966	0.960	−0.006

B Reproducibility Pointers

The current implementation is a research prototype. The public repository contains the training configuration, search projection module, QK reconstruction path, exact sparse-attention path, Quest-style page baseline, and FAISS/HNSW correctness path. The relevant current checkpoints are:

- `checkpoints_block_d128/search_step_1000.pt`: clean six-layer block-causal result.
- `checkpoints_all36_d128_block/protected/search_step_500_keep.pt`: best all-36 checkpoint observed so far.
- `checkpoints_all32_d128_block_reserve_0_1_2_35/search_step_1000.pt`: current broad-substitution checkpoint.

References

[1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. arXiv:2004.05150, 2020.

[2] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. arXiv:2009.14794, 2020.

[3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Re. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. Advances in Neural Information Processing Systems, 2022.

[4] Jeff Johnson, Matthijs Douze, and Herve Jegou. Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 2019.

[5] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. International Conference on Learning Representations, 2020.

[6] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. RetrievalAttention: Accelerating long-context LLM inference via vector retrieval. arXiv:2409.10516, 2024.

[7] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2018.

[8] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. arXiv:1609.07843, 2016. Introduces the WikiText-103 benchmark.

[9] Qwen Team. Qwen3-4B-Instruct-2507 model card. Hugging Face, 2025.

[10] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. Transactions of the Association for Computational Linguistics, 2021.

[11] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. arXiv:2406.10774, 2024.

[12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. Advances in Neural Information Processing Systems, 2017.

[13] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native Sparse Attention: Hardware-aligned and natively trainable sparse attention. arXiv:2502.11089, 2025.

[14] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big Bird: Transformers for longer sequences. Advances in Neural Information Processing Systems, 2020.

[15] Marcel Butucea. ann-sparseattention repository. GitHub: unixsysdev/ann-sparseattention, 2026.